# DATASTOCK · REGISTRO DE PRUEBAS DE ACEPTACIÓN

UT2 · Acceso a datos relacionales mediante JDBC | 2.º DAM | Curso 2026–2027

*Plantilla oficial de registro de pruebas de aceptación del sistema DataStock para la fase relacional. Incluye los 20 casos de prueba de MySQL (CP-JDBC-01 a CP-JDBC-20) y los 3 casos de la actividad complementaria embebida SQLite (CP-SQLITE-01 a CP-SQLITE-03).*

| | |
|---|---|
| **Alumno/a** | Guillermo Beltrán Tabares |
| **Repositorio GitHub (URL)** | https://github.com/GuilleBT/DataStock |
| **Rama y etiqueta de entrega** | ut2 (etiqueta: UT2-v1.0) |
| **Fecha de ejecución y verificación** | 08/11/2026 |

## 1. Resumen de pruebas de aceptación (UT2)

| ID prueba | RF | Entrada / datos | Resultado esperado | Resultado obtenido | Estado |
|---|---|---|---|---|---|
| **CP-JDBC-01** | RF-JDBC-01 | Conexión estándar con db.properties | Muestra metadatos BD y conteo de productos | Conexión OK: MySQL 8.0.46, BD datastock, 0 productos | **SUPERADA** |
| **CP-JDBC-02** | RF-JDBC-01 | Contraseña errónea en db.properties | Error controlado en rojo sin revelar clave | Mensaje [ERROR] de acceso denegado (código 1045) sin abortar | **SUPERADA** |
| **CP-JDBC-03** | RF-JDBC-02 | Alta categoría (ID 1, Periféricos) y prov (ID 1) | Registra y lista en tabla con orden ID | Alta [OK] y listados ordenados por ID correctamente | **SUPERADA** |
| **CP-JDBC-04** | RF-JDBC-02 | Duplicar ID/nombre cat y baja con producto FK | Rechazo duplicado y bloqueo por clave foránea | Error duplicado controlado; baja impedida por FK producto | **SUPERADA** |
| **CP-JDBC-05** | RF-JDBC-03 | Alta TEC-001, 49.90€, stock 10, cat 1, prov 1 | Inserta producto con BigDecimal y PreparedStatement | Producto almacenado en MySQL con todos sus datos válidos | **SUPERADA** |
| **CP-JDBC-06** | RF-JDBC-03 | Modificar nombre y precio de TEC-001 | Actualiza campos; no permite modificar stock | Nombre y precio actualizados en MySQL; stock protegido | **SUPERADA** |
| **CP-JDBC-07** | RF-JDBC-03 | Duplicar TEC-001 o asignar categoría 999 | Rechaza inserción por duplicado o FK inexistente | [ERROR] Código duplicado y FK inexistente rechazados en Java | **SUPERADA** |
| **CP-JDBC-08** | RF-JDBC-03 | Baja producto sin movs y con movs históricos | Baja permitida sin movs; bloqueada con movs | Elimina producto sin movs; bloquea con movs por FK | **SUPERADA** |
| **CP-JDBC-09** | RF-JDBC-04 | Listar todos los | JOIN con categoría y proveedor ordenado | Tabla con nombres de cat/prov | **SUPERADA** |

| | | | | | |
|---|---|---|---|---|---|
| | | productos en MySQL | código | ordenada por código asc | |
| **CP-JDBC-10** | RF-JDBC-04 | Búsqueda LIKE con texto "Teclado" e inexistente | Muestra coincidencias o aviso informativo | Encuentra TEC-001; con texto inexistente muestra aviso | **SUPERADA** |
| **CP-JDBC-11** | RF-JDBC-04 | Filtro stock < 20 unidades | Productos con existencias estrictamente menores | Muestra TEC-001 (stock 10) y omite stock >= 20 | **SUPERADA** |
| **CP-JDBC-12** | RF-JDBC-05 | CALL registrar_movimiento (TEC-001, ENTRADA, 5) | Incrementa stock (10 -> 15) y crea movimiento | Stock anterior 10 -> posterior 15; registro en movimiento_stock | **SUPERADA** |
| **CP-JDBC-13** | RF-JDBC-05 | CALL registrar_movimiento (TEC-001, SALIDA, 4) | Decrementa stock (15 -> 11) y crea movimiento | Stock anterior 15 -> posterior 11; movimiento registrado | **SUPERADA** |
| **CP-JDBC-14** | RF-JDBC-05 | CALL registrar_movimiento (TEC-001, SALIDA, 20) | Error stock insuficiente; no altera stock ni tabla | Captura SIGNAL 45000: Stock insuficiente. Stock queda en 11 | **SUPERADA** |
| **CP-JDBC-15** | RF-JDBC-06 | Consultar historial completo de movimientos | Lista movimientos ordenados por fecha e ID | Muestra ID 1 (ENTRADA 5) e ID 2 (SALIDA 4) ordenados | **SUPERADA** |
| **CP-JDBC-16** | RF-JDBC-06 | Consultar movimientos de producto sin movs | Aviso informativo sin lanzar error ni excepción | Muestra "Sin movimientos registrados para el producto" | **SUPERADA** |
| **CP-JDBC-17** | RF-JDBC-07 | Importar lote memoria (2 válidos, 1 FK prov 3 ausente) | Inserta válidos, rechaza prov 3, transacción commit | Insertados: 2, Omitidos: 0, Rechazados: 1. Resumen en verde | **SUPERADA** |
| **CP-JDBC-18** | RF-JDBC-07 | Fallo SQL forzado durante lote de importación | Rollback automático; no persiste inserción parcial | Rollback ejecutado ante fallo; lote revertido íntegramente | **SUPERADA** |
| **CP-JDBC-19** | RF-JDBC-08 | Demo transacción A: dos categorías con commit() | Ambas categorías confirmadas y visibles con SELECT | Categorías 9901 y 9902 existen en MySQL tras commit | **SUPERADA** |
| **CP-JDBC-20** | RF-JDBC-08 | Demo transacción B: inserción y duplicado con rollback() | Revierte transacción; primera categoría no persiste | Categoría 9903 no existe tras rollback; integridad verificada | **SUPERADA** |
| **CP-SQLITE-01** | RA2-b | Ejecutar DemoSQLite por primera vez | Crea fichero .db, tabla e inserta 2 filas | Conexión OK, tabla lista, Insertados: 2 \| Existentes: 0 | **SUPERADA** |
| **CP-SQLITE-02** | RA2-b | Ejecutar DemoSQLite por segunda vez | Control de duplicados con INSERT OR IGNORE | Insertados: 0 \| Ya existentes: 2; sin filas duplicadas | **SUPERADA** |
| **CP-SQLITE-03** | RA2-b | Cierre de recursos y | Recursos liberados y | Fichero | **SUPERADA** |

| | | |
|---|---|---|
| persistencia fichero SQLite | archivo físico persistente | datos/sqlite/productos_demo.db (12 KB) persiste |

## 2. Ficha detallada de una prueba representativa

**Identificador:** CP-JDBC-12
**Requisito funcional:** RF-JDBC-05 · Registrar movimientos con procedimiento almacenado
**Objetivo:** Comprobar que una entrada de stock invoca atómicamente el procedimiento registrar_movimiento mediante CallableStatement, incrementa las existencias en producto y genera la fila correspondiente en movimiento_stock.
**Precondiciones:** Base de datos datastock activa en MySQL con tabla producto poblada y producto TEC-001 con stock inicial de 10 unidades.
**Datos de entrada:** Código: TEC-001 | Tipo: ENTRADA | Cantidad: 5 unidades | Confirmación: S
**Pasos ejecutados:** 1. Acceder al menú 7 (Gestión MySQL) y seleccionar opción 6 (Movimientos de stock).
2. Seleccionar opción 1 (Registrar una ENTRADA).
3. Introducir código "TEC-001", validar datos en pantalla (Stock actual: 10).
4. Introducir cantidad 5 y confirmar con "S".
5. Verificar la salida en consola y consultar el historial en opción 7.
**Resultado esperado:** Invocación correcta de CallableStatement, stock actualizado a 15 y nuevo registro en movimiento_stock con stock_anterior=10 y stock_posterior=15.
**Resultado obtenido:** Operación [OK] en verde. ID Movimiento: 1, Tipo: ENTRADA, Cantidad: 5, Stock anterior: 10 -> Stock posterior: 15. Comprobado en MySQL.
**Estado:** SUPERADA
**Evidencias:** Salida de consola validada en pruebas de integración, verificación en tabla MySQL y commit descriptivo en la rama ut2.

## 3. Incidencias detectadas durante el desarrollo y correcciones

| ID / Requisito | Problema detectado | Corrección aplicada | Resultado tras repetir |
|---|---|---|---|
| RF-JDBC-01 / Conexión | Advertencia de Connector/J sobre hilos pendientes (AbandonedConnectionCleanupThread) al salir de Maven exec:java. | Invocado AbandonedConnectionCleanupThread.checkedShutdown() en bloque de finalización de la aplicación. | Cierre limpio e instantáneo del comando Maven sin advertencias de hilos huérfanos. |
| RF-JDBC-07 / Importación | Riesgo de que un producto inválido cancele la importación de otros productos válidos del mismo lote. | Validación previa en dos fases: filtrado individual en Java y agrupación exclusiva de productos válidos para el bloque transaccional. | Los productos válidos se insertan y confirman en MySQL; los inválidos se notifican con detalle sin bloquear el resto del lote. |

## 4. Conclusión de la ejecución

**Número de pruebas ejecutadas: 23  Superadas: 23  No superadas: 0**

Observaciones finales: Se han implementado y superado con éxito todos los requisitos funcionales de la UT2 (RF-JDBC-01 al RF-JDBC-08) y los 3 casos de la actividad complementaria con SQLite embebido (CP-SQLITE-01 a CP-SQLITE-03). Las funcionalidades de la UT1 se mantienen 100% operativas. La aplicación respeta escrupulosamente las restricciones de integridad del script SQL, la seguridad de credenciales mediante db.properties y el control manual de transacciones JDBC.

Fecha: 8 de noviembre de 2026